Module 6 – **Planning**

How can an intelligent agent decide which actions to perform, and in what order, to achieve a goal?

Outline

1. Basics on Planning
   - The Problem
   - The PDDL Language
2. Search Strategies and Heuristics
   - Forward and Backward Search
   - Heuristics
3. Planning Graphs, Heuristics and Graphplan
   - Planning Graphs
   - The Graphplan Algorithm

Automated Planning (aka “Planning”)

Meaning: Planning is finding a sequence of actions to move from the current state to the desired goal.

- **Planning as an application:** Planning itself is the main task. Example: Finding a path for a robot to reach its destination.
- **Planning as part of an application:** Planning is one component of a larger system. Example: A self-driving car plans its route while driving.
- **Applications:** Robotics, job scheduling, design, and manufacturing.

Classical planning: Similar to problem-solving in AI, but states and actions are represented in a structured way.

Assumptions of classical planning:

- Fully observable: The agent knows the current state.
- Deterministic: Each action has a predictable outcome.
- Static: The world does not change unexpectedly while planning.
- Single agent: Only one agent is responsible for planning and acting.

Key idea: AI planning determines which actions to perform and in what order to achieve a goal.

A Language for Planning: PDDL

PDDL: A Language for Planning

PDDL (Planning Domain Definition Language) is a language used to describe planning problems in Artificial Intelligence.

1. **State:** Describes the current situation of the world.
    1. Example: The robot is in Room A.
2. **Initial State and Goal State:** The initial state tells where the agent starts, and the goal state tells what it must achieve.
    1. Example: Initial state — Robot is in Room A. Goal state — Robot is in Room B.
3. **Actions:** Describe the operations the agent can perform.
    1. Example: Move the robot from Room A to Room B.
4. **Preconditions:** Specify the conditions that must be true before an action can be performed.
    1. Example: The robot must be in Room A before it can move from Room A to Room B.
5. **Effects:** Describe what changes after an action is performed.
    1. Example: After moving, the robot is no longer in Room A and is now in Room B.

PDDL: Example

The main idea: How do we describe an airplane's flying action so that an AI planner can understand when it can fly and what changes after it flies? Suppose an airplane named P1 is at San Francisco Airport (SFO). We want it to fly to John F. Kennedy Airport (JFK).

Consider : **Fly(p, from, to)**

Preconditions

- `Plane(p)` — p is a plane.
- `Airport(from)` — the starting location is an airport.
- `Airport(to)` — the destination is an airport.
- `At(p, from)` — the plane is currently at the starting airport.

Effects

- `¬At(p, from)` — the plane is no longer at the starting airport.
- `At(p, to)` — the plane is now at the destination airport.

PDDL: Example

The action `Fly(p, from, to)` means: fly plane `p` from one airport to another.

Before flying, the planner checks that the object is a plane, both locations are airports, and the plane is at the starting airport. After flying, the plane's location changes.

Action schema:

$Action(Fly(p, from, to),$
$\quad PRECOND : Plane(p) \wedge Airport(from) \wedge Airport(to) \wedge At(p, from)$
$\quad EFFECT \quad : \neg At(p, from) \wedge At(p, to))$

Action instantiation:

$Action(Fly(P_1, SFO, JFK),$
$\quad PRECOND : Plane(P_1) \wedge Airport(SFO) \wedge Airport(JFK) \wedge At(P_1, SFO)$
$\quad EFFECT \quad : \neg At(P_1, SFO) \wedge At(P_1, JFK))$

A Language for Planning: PDDL [cont.]

- Precondition: must hold to ensure the action can be executed
  - defines the states in which the action can be executed
  - action is applicable in state s if the preconditions are satisfied by s
- Effect: represent the effects of the action on the world
  - defines the result of executing the action
- Add list (ADD(a)): (the fluents in) the positive literals in the action's effects
  - ex: $\{At(p, to)\}$
- Delete list (DEL(a)): (the fluents in) the negative literals in the action's effects
  - ex: $\{At(p, from)\}$
- Result of action a in state s: RESULT(s,a)$\stackrel{def}{=}$(s\DEL(a) $\cup$ ADD(a))
  - Start with the current state.
  - Remove the facts in the Delete List.
  - Add the facts in the Add List.
  - ex: $Fly(P_1, SFO, JFK) \Longrightarrow$ remove $At(P_1, SFO)$, add $At(P_1, JFK)$

PDDL: Example [cont.]

- Action schema:
  $Action(Fly(p, from, to),$
  $PRECOND : Plane(p) \wedge Airport(from) \wedge Airport(to) \wedge At(p, from)$
  $EFFECT : \neg At(p, from) \wedge At(p, to))$
- Action instantiation:
  $Action(Fly(P_1, SFO, JFK),$
  $PRECOND : Plane(P_1) \wedge Airport(SFO) \wedge Airport(JFK) \wedge At(P_1, SFO)$
  $EFFECT : \neg At(P_1, SFO) \wedge At(P_1, JFK))$
- $s : At(P_1, SFO) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport(JFK) \wedge ...$

$\Longrightarrow$ $s' : At(P_1, JFK) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport(JFK) \wedge ...$

A Language for Planning: PDDL [cont.]

Planning as a search problem

All components of a search problem

- an initial state
- an ACTIONS function
- a RESULT function
- and a goal test

Example: Air Cargo Transport : Transport two cargo items between SFO and JFK.

Initial State

- Cargo C1 and airplane P1 are at SFO.
- Cargo C2 and airplane P2 are at JFK.

Goal State

- Cargo C1 must reach JFK.
- Cargo C2 must reach SFO.

Available Actions

1. **Load:** Put cargo into an airplane at an airport.
2. **Fly:** Move an airplane from one airport to another.
3. **Unload:** Remove cargo from an airplane at an airport.

One Possible Solution

1. Load C1 into P1 at SFO.
2. Fly P1 from SFO to JFK.
3. Unload C1 at JFK.
4. Load C2 into P2 at JFK.
5. Fly P2 from JFK to SFO.
6. Unload C2 at SFO.

Key Takeaway: A planning algorithm finds an appropriate sequence of actions that transforms the initial state into a state satisfying all the goals.

Example: Air Cargo Transport using PDDL

$$
\begin{aligned}
&Init(At(C_1, SFO) \land At(C_2, JFK) \land At(P_1, SFO) \land At(P_2, JFK) \\
&\quad \land Cargo(C_1) \land Cargo(C_2) \land Plane(P_1) \land Plane(P_2) \\
&\quad \land Airport(JFK) \land Airport(SFO)) \\
&Goal(At(C_1, JFK) \land At(C_2, SFO)) \\
&Action(Load(c, p, a), \\
&\quad \text{PRECOND: } At(c, a) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a) \\
&\quad \text{EFFECT: } \neg At(c, a) \land In(c, p)) \\
&Action(Unload(c, p, a), \\
&\quad \text{PRECOND: } In(c, p) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a) \\
&\quad \text{EFFECT: } At(c, a) \land \neg In(c, p)) \\
&Action(Fly(p, from, to), \\
&\quad \text{PRECOND: } At(p, from) \land Plane(p) \land Airport(from) \land Airport(to) \\
&\quad \text{EFFECT: } \neg At(p, from) \land At(p, to))
\end{aligned}
$$

(© S. Russell & P. Norwig, AIMA)

One solution: [*Load*(C_1, P_1, *SFO*), *Fly*(P_1, *SFO*, *JFK*), *Unload*(C_1, P_1, *JFK*), *Load*(*C2*, *P2*, *JFK*), *Fly*(*P2*, *JFK*, *SFO*), *Unload*(*C2*, *P2*, *SFO*)]

Example: Spare Tire Problem

Initial State

- The flat tire is on the axle.
- The spare tire is in the trunk.

Goal

Install the spare tire on the axle.

Available Actions

1. **Remove(obj, loc):** Remove a tire from its current location and place it on the ground.
2. **PutOn(t, Axle):** Install a tire on the axle if the axle is free and the tire is on the ground.
3. **LeaveOvernight:** In this example, leaving the car overnight causes the tires to disappear.

One Possible Solution

1. Remove the flat tire from the axle.
2. Remove the spare tire from the trunk.
3. Put the spare tire on the axle.

Key Takeaway

A planning algorithm identifies the actions and their correct order to achieve a goal while satisfying the action preconditions.

Example: Spare Tire Problem using PDDL

$$
\begin{aligned}
&Init(Tire(Flat) \wedge Tire(Spare) \wedge At(Flat, Axle) \wedge At(Spare, Trunk))\\
&Goal(At(Spare, Axle))\\
&Action(Remove(obj, loc),\\
&\quad \text{PRECOND: } At(obj, loc)\\
&\quad \text{EFFECT: } \neg At(obj, loc) \wedge At(obj, Ground))\\
&Action(PutOn(t, Axle),\\
&\quad \text{PRECOND: } Tire(t) \wedge At(t, Ground) \wedge \neg At(Flat, Axle)\\
&\quad \text{EFFECT: } \neg At(t, Ground) \wedge At(t, Axle))\\
&Action(LeaveOvernight,\\
&\quad \text{PRECOND:}\\
&\quad \text{EFFECT: } \neg At(Spare, Ground) \wedge \neg At(Spare, Axle) \wedge \neg At(Spare, Trunk)\\
&\qquad\qquad \wedge \neg At(Flat, Ground) \wedge \neg At(Flat, Axle) \wedge \neg At(Flat, Trunk))
\end{aligned}
$$

(© S. Russell & P. Norwig, AIMA)

(We assume that the car is parked in a particularly bad neighborhood, so that the effect of leaving it overnight is that the tires disappear.)

One solution: [*Remove*(*Flat, Axle*), *Remove*(*Spare, Trunk*), *PutOn*(*Spare, Axle*)]

Example: Blocks World

Objective: Rearrange blocks into a desired arrangement using a sequence of actions.

Rules

- Move only one block at a time.
- A block can be moved only when its top is clear.
- Place a block either on the table or on another clear block.

One Possible Solution

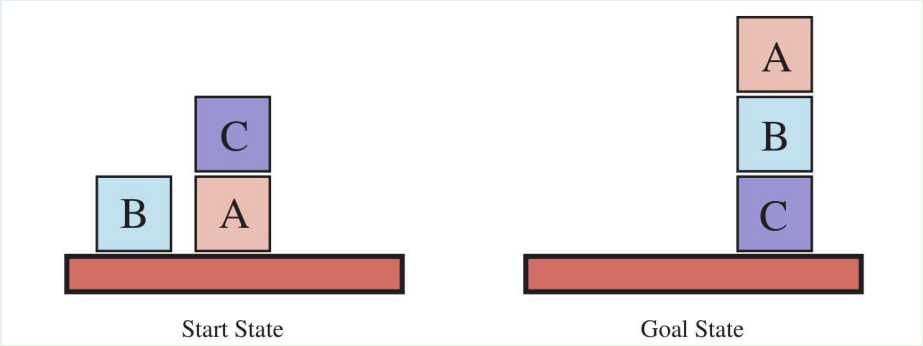
1. `MoveToTable(C, A)` — Move C from A onto the table.
2. `Move(B, Table, C)` — Place B on C.
3. `Move(A, Table, B)` — Place A on B.

Final Arrangement

- A is on B.
- B is on C.
- C is on the table.

Key Takeaway: AI planning determines the correct sequence of actions to transform the initial arrangement into the goal arrangement.

Example: Blocks World using PDDL



$$
\begin{aligned}
&Init(On(A, Table) \wedge On(B, Table) \wedge On(C, A) \\
&\quad \wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Clear(B) \wedge Clear(C)) \\
&Goal(On(A, B) \wedge On(B, C)) \\
&Action(Move(b, x, y), \\
&\quad \text{PRECOND: } On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge \\
&\qquad (b \neq x) \wedge (b \neq y) \wedge (x \neq y), \\
&\quad \text{EFFECT: } On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y)) \\
&Action(MoveToTable(b, x), \\
&\quad \text{PRECOND: } On(b, x) \wedge Clear(b) \wedge Block(b) \wedge (b \neq x), \\
&\quad \text{EFFECT: } On(b, Table) \wedge Clear(x) \wedge \neg On(b, x))
\end{aligned}
$$

(© S. Russell & P. Norwig, AIMA)

One solution: [*MoveToTable*(*C, A*), *Move*(*B, Table, C*), *Move*(*A, Table, B*)]

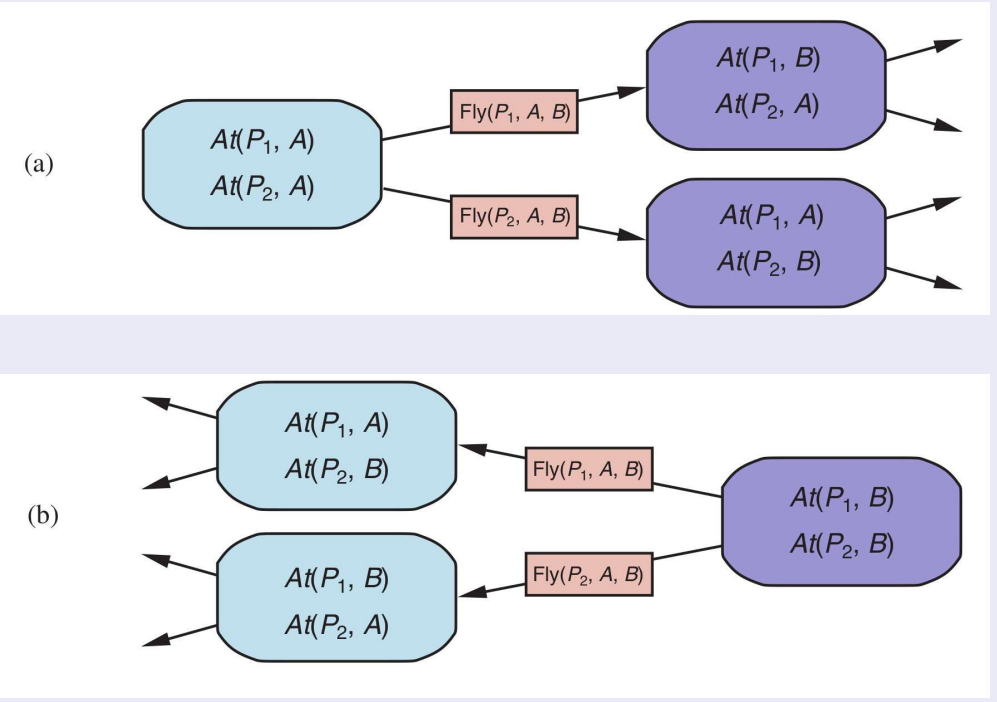
Outline

1. Basics on Planning
   - The Problem
   - The PDDL Language
2. Search Strategies and Heuristics
   - Forward and Backward Search
   - Heuristics
3. Planning Graphs, Heuristics and Graphplan
   - Planning Graphs
   - The Graphplan Algorithm

Two Main Approaches

(a) Forward search (aka progression search)
- start in the initial state
- use actions to search forward for a goal state

(b) Backward search (aka regression search)
- start from goals
- use reverse actions to search forward for the initial state



(© S. Russell & P. Norwig, AIMA)

Forward Search

- Forward search (aka progression search)
  - choose actions whose preconditions are satisfied
  - add positive effects, delete negative
- Goal test: does the state satisfy the goal?
- Step cost: each action costs 1

$\Longrightarrow$ We can use any of the search algorithms from Ch. 03, 04

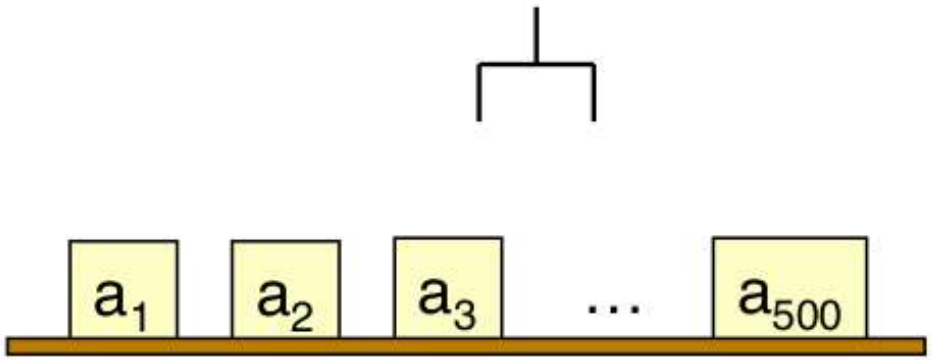
  - need keeping track of the actions used to reach the goal
- Breadth-first and best-first
  - Sound: if they return a plan, then the plan is a solution
  - Complete: if a problem has a solution, then they will return one
  - Require exponential memory wrt. solution length! $\Longrightarrow$ unpractical
- Depth-first search and greedy search
  - Sound
  - Not complete
    - may enter in infinite loops
    - (classical planning only): made complete by loop-checking
  - Require linear memory wrt. solution length

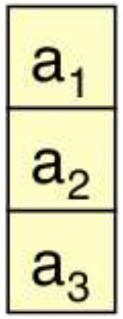
Branching Factor of Forward Search

- Planning problems can have huge state spaces
- Forward search can have a very large branching factor
  - ex: *pickup*(a_1), *pickup*(a_2), ..., *pickup*(a_{500})

$\Longrightarrow$ Forward-search can waste time trying lots of irrelevant actions

$\Longrightarrow$ Need a good heuristic to guide the search





(© of Dana Nau, CMSC21, U. Maryland, Licensed under Creative Commons)

Backward Search (aka Regression or Relevant-States)

- Predecessor (sub)goal g' of ground goal g via ground action a:
  $Pos(g') \stackrel{\text{def}}{=} (Pos(g) \setminus Add(a)) \cup Pos(Precond(a))$
  $Neg(g') \stackrel{\text{def}}{=} (Neg(g) \setminus Del(a)) \cup Neg(Precond(a))$
- Note: Both g and g' represent many states
  - irrelevant ground atoms unassigned

- Consider the goal $At(C_1, SFO) \wedge At(C_2, JFK)$
- Consider the ground action:
  $Action(Unload(C_1, P_1, SFO),$
  $PRECOND : In(C_1, P_1) \wedge At(P_1, SFO) \wedge Cargo(C_1) \wedge Plane(P_1) \wedge Airport(SFO)$
  $EFFECT : At(C_1, SFO) \wedge \neg In(C_1, P_1))$
- This produces the sub-goal g':
  $In(C_1, P_1) \wedge At(P_1, SFO) \wedge Cargo(C_1) \wedge Plane(P_1) \wedge Airport(SFO) \wedge At(C_2, JFK)$
- Both g' and g represent many states
  - e.g. truth value of $In(C_3, P_2)$ irrelevant

Backward Search [cont.]

- Idea: deal with partially un-instantiated actions and states
  - avoid unnecessary instantiations

  $\Longrightarrow$ no need to produce a goal for every possible instantiation
- use the most general unifier $\Longrightarrow$ compute weakest precondition
- standardize action schemata first (rename vars into fresh ones)

- Consider the goal $At(C_1, SFO) \wedge At(C_2, JFK)$
- Consider the partially-instantiated action:
  $Action(Unload(C_1, p', SFO),$
  $PRECOND : In(C_1, p') \wedge At(p', SFO) \wedge Cargo(C_1) \wedge Plane(p') \wedge Airport(SFO)$
  $EFFECT : At(C_1, SFO) \wedge \neg In(C_1, p'))$
- This produces the sub-goal g':
  $In(C_1, p') \wedge At(p', SFO) \wedge Cargo(C_1) \wedge Plane(p') \wedge Airport(SFO) \wedge At(C_2, JFK)$
- Represents states with all possible planes

  $\Longrightarrow$ no need to produce a subgoal for every plane $P_1, P_2, P_3, ...$

Backward Search [cont.]

Which action to choose?

- Relevant action: could be the last step in a plan for goal g
  - at least one of the action's effects (positive or negative) must unify with an element of the goal
  - must not undo desired literals of the goal
- Consistent action: must not undo desired literals of the goal
  - inconsistent actions are also non-relevant

- Ex: consider the goal $At(C_1, SFO) \wedge At(C_2, JFK)$
  - $Action(Unload(C_1, p', SFO), ...)$ is relevant (previous example)
  - $Action(Unload(C_3, p', SFO), ...)$ is not relevant
  - $Action(Load(C_2, p', JFK), ...)$ is not consistent $\Longrightarrow$ is not relevant

\+ B.S. typically keeps the branching factor lower than F.S.

\- B.S. reasons with state sets
$\Longrightarrow$ makes it harder to come up with good heuristics

- Most planners work with forward search plus heuristics

Heuristics for (Forward-Search) Planning

A^* for Planning

- Recall: A^* is a best-first algorithm which
  - uses an evaluation function f(s) = g(s) + h(s),
  - g(s): (exact) cost to reach s
  - h(s): admissible (optimistic) heuristics
    (never overestimates the distance to the goal)
- A technique for admissible heuristics: problem relaxation
  $\Longrightarrow$ h(s): the exact cost of a solution to the relaxed problem
- Forms of problem relaxation exploiting problem structure
  - Add arcs to the search graph $\Longrightarrow$ make it easier to search
    - ignore-preconditions heuristics
    - ignore-delete-lists heuristics
  - Clustering nodes (aka state abstraction) $\Longrightarrow$ reduce search space
    - ignore less-relevant fluents

Outline

1. Basics on Planning
   - The Problem
   - The PDDL Language
2. Search Strategies and Heuristics
   - Forward and Backward Search
   - Heuristics
3. Planning Graphs, Heuristics and Graphplan
   - Planning Graphs
   - The Graphplan Algorithm

Planning Graph

Definition: A planning graph is a directed graph organized into levels to represent possible states and actions.

1. **Initial State (S_0):** Contains all facts that are true in the initial state.
2. **Action Level (A_0):** Contains actions whose preconditions are satisfied in S_0.
3. **Next State (S_1):** Contains the effects of actions in A_0.
4. **Repeat:** Continue adding action and state levels until the graph levels off.

Additional Concepts

- **No-op actions:** Represent facts that remain unchanged.
- **Mutex (Mutual Exclusion):** Indicates actions or facts that cannot occur together.

Key Point: A planning graph helps estimate how many steps may be needed to reach the goal and supports the Graphplan algorithm.

Building a Planning Graph

1. **Create Initial State (S_0):** Add all facts that are true in the initial state.
2. **Create Action Level (A_0):** Add actions whose preconditions are satisfied in S_0.
3. **Add No-op Actions:** Include actions that allow existing facts to remain unchanged.
4. **Create Next State (S_1):** Add the effects of actions from A_0.
5. **Identify Mutex:** Mark actions or facts that cannot occur together.
6. **Repeat:** Continue creating action and state levels until the graph levels off or a specified limit is reached.

Key Terms

- **S_i:** State or fact level.
- **A_i:** Action level.
- **Mutex:** Mutual exclusion between incompatible actions or facts.
- **No-op:** An action that preserves a fact without changing it.

Key Point: A planning graph expands level by level to represent possible actions and their effects.

Planning in Artificial Intelligence: The Cake Example

This example explains how an AI planning system decides which actions to perform to achieve a goal. It uses a simple situation: you have a cake, you want to eat it, but you also want to have a cake afterward.
The interesting part is that the AI must choose the correct sequence of actions: Eat the existing cake, then bake a new cake.

1. Understand the problem

Suppose an intelligent agent is given the following information:

- Initially, a cake is available.
- The goal is to have a cake and to have eaten a cake.
- Eating a cake removes the cake from the available inventory but records that it has been eaten.
- Baking a cake creates a cake, but baking is possible only when no cake is currently available.

The agent must find a sequence of actions that satisfies both parts of the goal.

AI Planning: The Cake Example

- **Initial State:** Have a cake; no cake has been eaten.
- **Goal State:** Have a cake AND have eaten a cake.

- **Action 1 – Eat(Cake):** Precondition: Have(Cake). Effects: ¬Have(Cake), Eaten(Cake).
- **Action 2 – Bake(Cake):** Precondition: ¬Have(Cake). Effect: Have(Cake).

- **Plan:** Eat the existing cake → Bake a new cake.

- **Planning Graph:** State levels (S_0, S_1, S_2) represent states; action levels (A_0, A_1) represent actions. No-ops preserve facts, and mutex links indicate mutually exclusive actions or states.

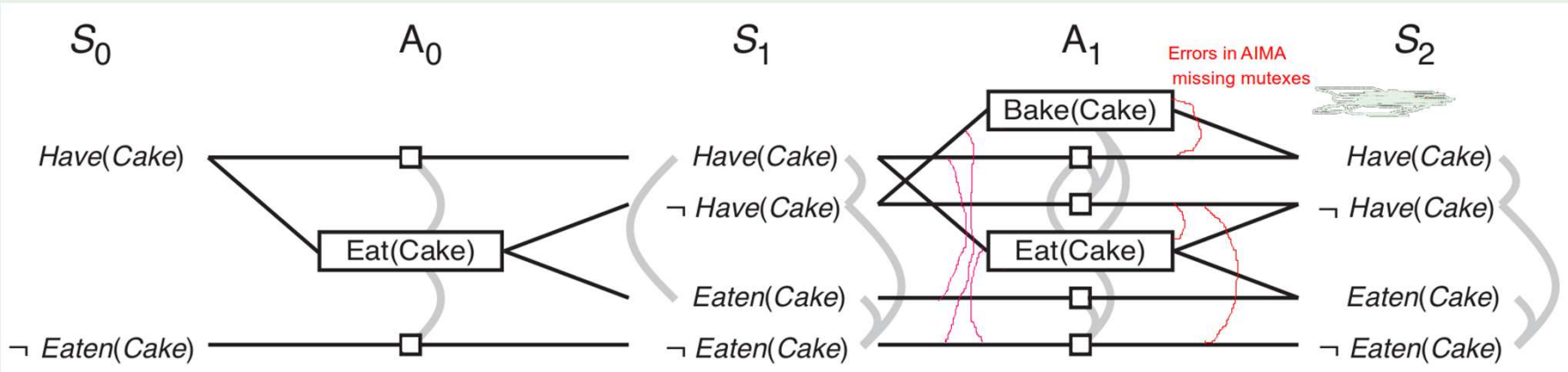
Key Idea: AI planning finds a valid sequence of actions that satisfies all goal conditions. It alternates between state levels (\(S_0,S_1,S_2\)) and action levels (\(A_0,A_1\)).

Planning Graph: Example

$Init(Have(Cake))$
$Goal(Have(Cake) \wedge Eaten(Cake))$
$Action(Eat(Cake)$
PRECOND: $Have(Cake)$
EFFECT: $\neg\, Have(Cake) \wedge Eaten(Cake))$
$Action(Bake(Cake)$
PRECOND: $\neg\, Have(Cake)$
EFFECT: $Have(Cake))$

You would like to eat your cake and still have a cake. Fortunately, you can bake a new one.

Rectangles indicate actions
Small squares persistence actions (**no-ops**)
Straight lines indicate preconditions and effects
Mutex links are shown as curved gray lines



(© S. Russell & P. Norwig, AIMA)

Mutex Computation: Example

Example: Cake Problem

Two facts may be mutex if they cannot be achieved together.

1. **Inconsistent Support:** One fact is the negation of the other.
    1. Example: `Have(Cake)` and `¬Have(Cake)` cannot both be true.
2. **All Ways Are Mutually Exclusive:** If every possible way of achieving two facts involves mutex actions, those facts are also mutex.

Example:

- `Have(Cake)`: Cake is available.
- `Eaten(Cake)`: Cake has been eaten.

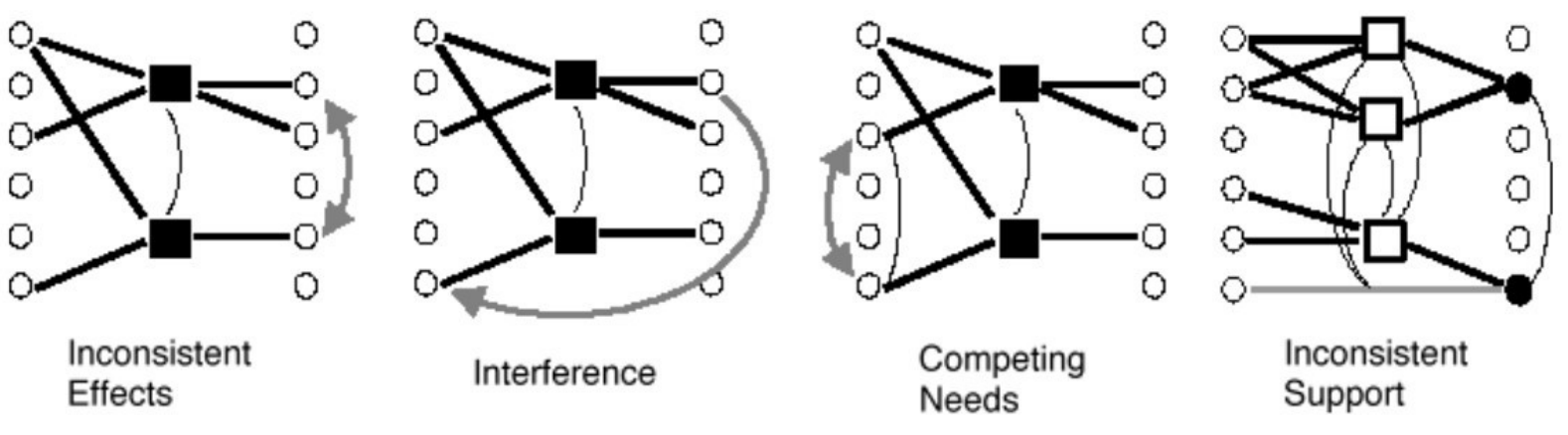
At a state level, `Have(Cake)` and `Eaten(Cake)` can be mutex if all ways of achieving them together are mutually exclusive.

Key Point: Literal mutex helps identify facts that cannot be achieved together at the same level of a planning graph.

Mutex Computation

- Two actions at the same action-level have a mutex relation if
  - Inconsistent effects: an effect of one negates an effect of the other
  - Interference: one deletes a precondition of the other
  - Inconsistent preconditions (aka competing needs): they have mutually exclusive preconditions
- Otherwise they don't interfere with each other

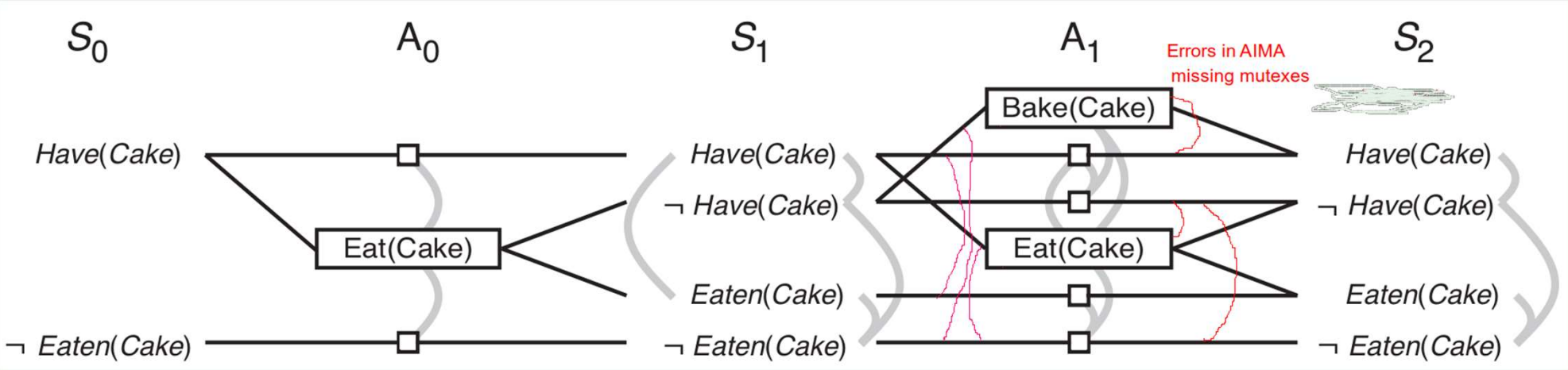
  $\Longrightarrow$ both may appear in a solution plan
- Two literals at the same state-level have a mutex relation if
  - inconsistent support: one is the negation of the other
  - all ways of achieving them (including no-ops) are pairwise mutex



(© of Dana Nau, CMSC21, U. Maryland, Licensed under Creative Commons)

Mutex Computation: Example

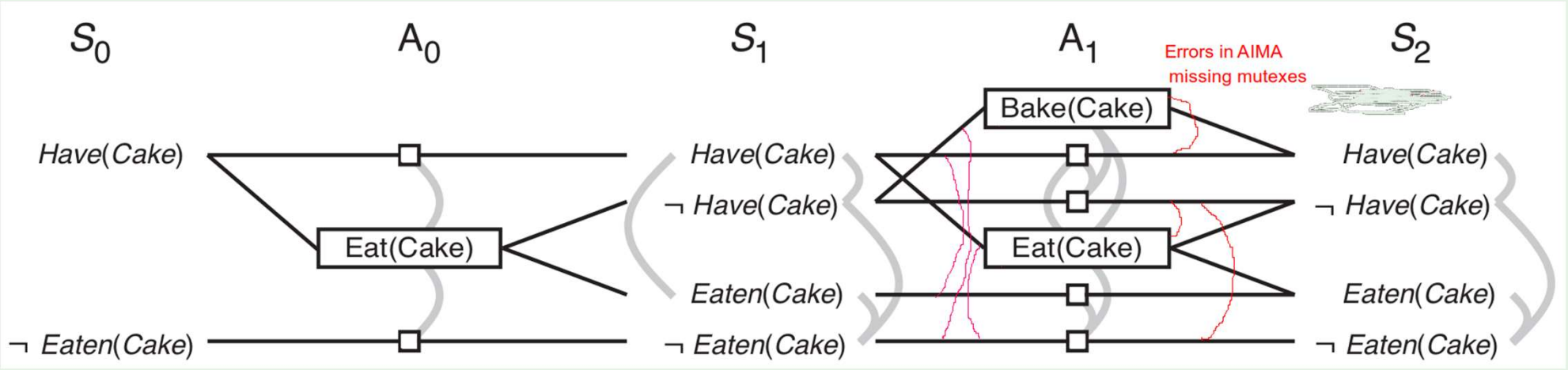
- Two actions at the same action-level have a mutex relation if
  - Inconsistent effects: an effect of one negates an effect of the other
    ex: persistence of *Have*(*Cake*), *Eat*(*Cake*) have competing effects
    ex: *Bake*(*Cake*), *Eat*(*Cake*) have competing effects
  - Interference: one deletes a precondition of the other
    ex: *Eat*(*Cake*) interferes with the persistence of *Have*(*Cake*)
  - Inconsistent preconditions (aka competing needs): they have mutually exclusive preconditions
    ex: *Bake*(*Cake*) and *Eat*(*Cake*)



(© S. Russell & P. Norwig, AIMA)

Mutex Computation: Example [cont.]

- Two literals at the same state-level have a mutex relation if
  - inconsistent support: one is the negation of the other
    ex.: *Have*(*Cake*), ¬*Have*(*Cake*)
  - all ways of achieving them are pairwise mutex
    ex.: (S_1): *Have*(*Cake*) in mutex with *Eaten*(*Cake*)
    because persistence of *Have*(*Cake*), *Eat*(*Cake*) are mutex



(© S. Russell & P. Norwig, AIMA)

Outline

1. Basics on Planning
   - The Problem
   - The PDDL Language
2. Search Strategies and Heuristics
   - Forward and Backward Search
   - Heuristics
3. Planning Graphs, Heuristics and Graphplan
   - Planning Graphs
   - The Graphplan Algorithm

Graphplan Algorithm in Artificial Intelligence

Definition: Graphplan is an AI planning algorithm that constructs a planning graph to find a sequence of actions that achieves a given goal.

Steps of Graphplan:

1. **Initialize:** Start with the initial state (S_0).
2. **Expand:** Add applicable actions and their effects, creating alternating state and action levels.
3. **Identify Mutex:** Mark mutually exclusive actions and states.
4. **Check Goal:** Verify that all goal conditions appear together without mutex conflicts.
5. **Extract Plan:** Search backward through the graph to find a valid sequence of actions. If unsuccessful, expand the graph further.

Key Idea: Graphplan expands the planning graph forward and extracts a valid plan backward.

The Graphplan Algorithm

- A strategy for extracting a plan from the planning graph
- Repeatedly adds a level to a planning graph (EXPAND-GRAPH)
- If all the goal literals occur in last level and are non-mutex
  - search for a plan that solves the problem (EXTRACT-SOLUTION)
  - if that fails, expand another level and try again (and add ⟨*goal, level*⟩ as nogood)
- If graph and nogoods have both leveled off then return failure
- Depends on EXPAND-GRAPH & EXTRACT-SOLUTION

function GRAPHPLAN(*problem*) **returns** solution or failure

  graph ← INITIAL-PLANNING-GRAPH(*problem*)
  goals ← CONJUNCTS(*problem*.GOAL)
  nogoods ← an empty hash table
  for $t = 0$ **to** ∞ **do**
    if *goals* all non-mutex in S_t of *graph* **then**
      solution ← EXTRACT-SOLUTION(*graph*, *goals*, NUMLEVELS(*graph*), *nogoods*)
      if *solution* ≠ *failure* **then return** *solution*
    if *graph* and *nogoods* have both leveled off **then return** *failure*
    graph ← EXPAND-GRAPH(*graph*, *problem*)

typo in AIMA book

(© S. Russell & P. Norwig, AIMA)

Graphplan: Spare Tire Problem

Goal: Place the spare tire on the axle.

1. **Create Initial Layer (S_0):** Add the facts from the initial state and the relevant facts implied by the closed-world assumption.
2. **Check the Goal:** `At(Spare, Axle)` is not present in S_0.
3. **Expand the Graph:** Use `EXPAND-GRAPH` to add action levels, state levels, and mutex relationships.
4. **Check Again:** If the goal is still absent, expand another level.
5. **Extract a Plan:** When the goal appears and the required goal literals are not mutex, call `EXTRACT-SOLUTION` to search for a valid plan.

Key Point: Graphplan expands the planning graph until it can attempt to extract a plan that achieves the goal.

Graphplan Algorithm: Spare Tire Problem

Objective: Replace a flat tire with a spare tire using a sequence of valid actions.

- **Initial State (S_0):** Spare tire is in the trunk; flat tire is on the axle.
- **Actions (A_0, A_1):**
  - `Remove(Spare, Trunk)`: Removes the spare tire from the trunk.
  - `Remove(Flat, Axle)`: Removes the flat tire from the axle.
  - `LeaveOvernight`: Leaves the car overnight.
  - `PutOn(Spare, Axle)`: Places the spare tire on the axle.
- **State Levels (S_0, S_1, S_2):** Show how the possible facts about tire locations change as actions are applied.
- **Mutex Links:** Identify mutually exclusive actions or facts that cannot hold together at the same planning level.
- **Plan:** Remove the spare from the trunk → Remove the flat tire from the axle → Put the spare tire on the axle.

Key Idea: Graphplan builds a planning graph, identifies mutex relationships, and extracts a valid sequence of actions to achieve the goal.

[Recall] Example: Spare Tire Problem

$$
\begin{aligned}
&Init(Tire(Flat) \wedge Tire(Spare) \wedge At(Flat, Axle) \wedge At(Spare, Trunk))\\
&Goal(At(Spare, Axle))\\
&Action(Remove(obj, loc),\\
&\quad \text{PRECOND: } At(obj, loc)\\
&\quad \text{EFFECT: } \neg At(obj, loc) \wedge At(obj, Ground))\\
&Action(PutOn(t, Axle),\\
&\quad \text{PRECOND: } Tire(t) \wedge At(t, Ground) \wedge \neg At(Flat, Axle)\\
&\quad \text{EFFECT: } \neg At(t, Ground) \wedge At(t, Axle))\\
&Action(LeaveOvernight,\\
&\quad \text{PRECOND:}\\
&\quad \text{EFFECT: } \neg At(Spare, Ground) \wedge \neg At(Spare, Axle) \wedge \neg At(Spare, Trunk)\\
&\qquad\qquad \wedge \neg At(Flat, Ground) \wedge \neg At(Flat, Axle) \wedge \neg At(Flat, Trunk))
\end{aligned}
$$

(© S. Russell & P. Norwig, AIMA)

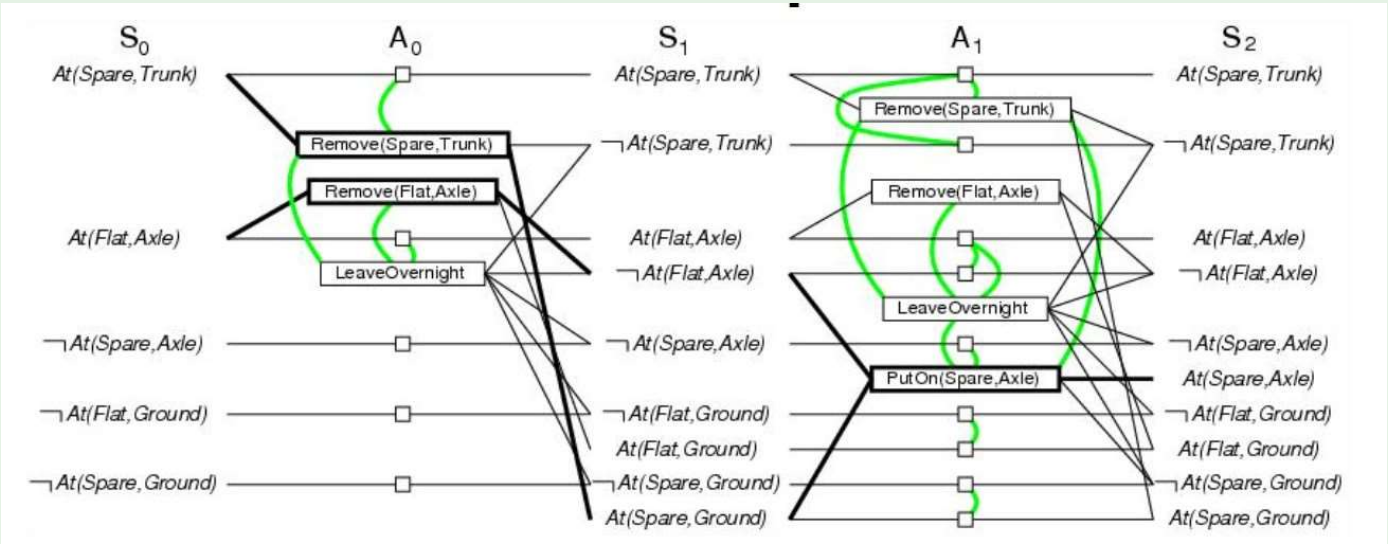
(We assume that the car is parked in a particularly bad neighborhood, so that the effect of leaving it overnight is that the tires disappear.)

One solution: [*Remove*(*Flat, Axle*), *Remove*(*Spare, Trunk*), *PutOn*(*Spare, Axle*)]

Graphplan: Example Spare Tire problem

- Initial plan 5 literals from initial state and the Closed-World-Assumption literals (S_0).
  - fixed literals (e.g. *Tire*(*Flat*)) ignored here
  - irrelevant literals ignored here
- Goal *At*(*Spare, Axle*) not present in S_0

  $\Longrightarrow$ no need to call EXTRACT-SOLUTION
- Graph and nogoods not leveled off $\Longrightarrow$ invoke EXPAND-GRAPH

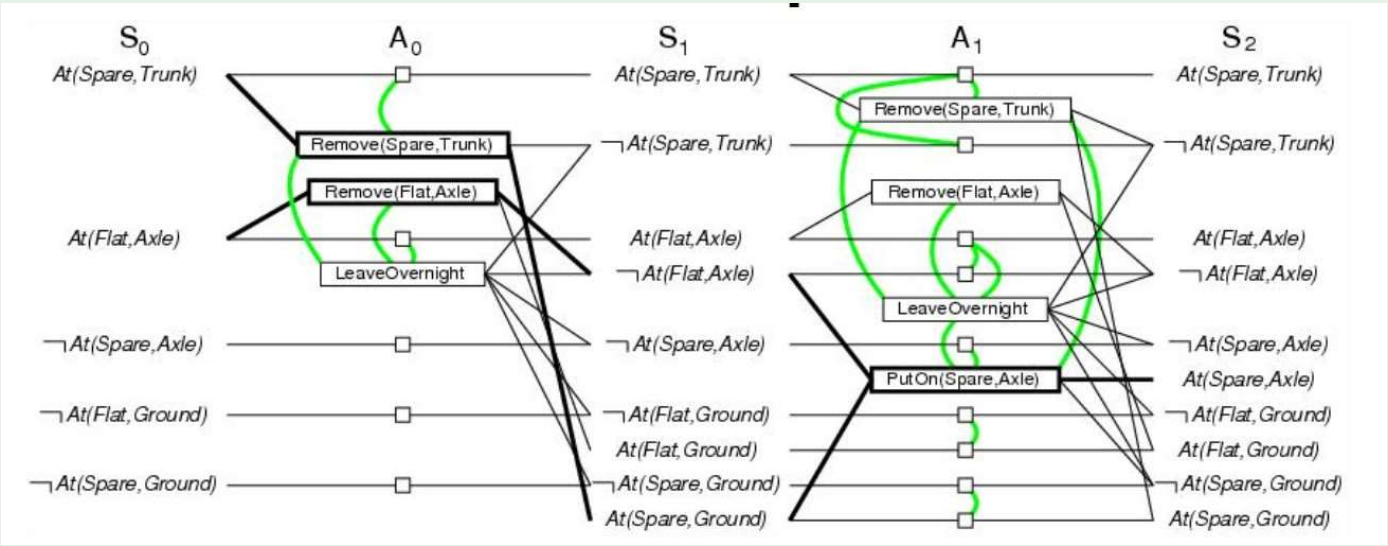


(© S. Russell & P. Norwig, AIMA) (inter-fluent mutexes omitted for readability)

Graphplan: Example [cont.] Spare Tire problem

- Invoke EXPAND-GRAPH
  - add actions A_0, persistence actions and mutexes
  - add fluents S_1 and mutexes
- Goal *At*(*Spare, Axle*) not present in S_1

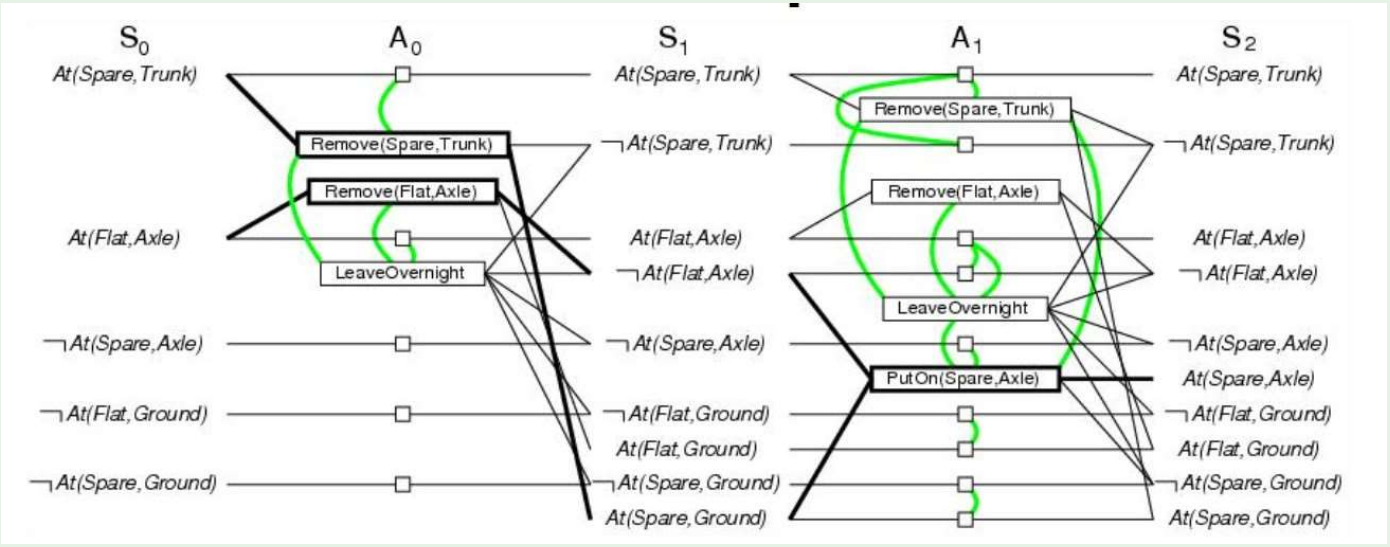
  $\Longrightarrow$ no need to call EXTRACT-SOLUTION
- Graph and nogoods not leveled off $\Longrightarrow$ invoke EXPAND-GRAPH



(© S. Russell & P. Norwig, AIMA) (inter-fluent mutexes omitted for readability)

Graphplan: Example [cont.] Spare Tire problem

- Invoke EXPAND-GRAPH
  - add actions A_1, persistence actions and mutexes
  - add fluents S_2 and mutexes
- Goal *At*(*Spare, Axle*) present in S_2
  - call EXTRACT-SOLUTION
- Solution found!



(© S. Russell & P. Norwig, AIMA) (inter-fluent mutexes omitted for readability)

Mutex Conditions in Graphplan

Mutex (Mutually Exclusive) means that two actions or two facts cannot occur together at the same level of a planning graph.
In Graphplan, there are two types of mutex relationships:

1. Action mutex: Two actions cannot be performed together at the same action level.
2. Proposition mutex: Two facts cannot be true together at the same state level.

1. Action mutex: Three conditions

2. Proposition mutex: Two conditions

Two facts are mutex when they cannot both be achieved at the same state level.

- Negation: One fact is the negation of the other. For example, `At(Spare, Trunk)` and `¬At(Spare, Trunk)`.
- Inconsistent support: Every pair of actions that could produce the two facts is mutually exclusive.

Action mutex: Three conditions

Condition	Meaning	Example
Inconsistent effects	One action makes a fact true, while another makes it false.	Remove(Spare, Trunk) and an action that puts the spare back in the trunk.
Interference	One action deletes a precondition or effect required by another action.	One action removes the spare from the trunk while another requires it to remain there.
Competing needs	The actions have preconditions that are mutually exclusive.	One action requires At(Spare, Trunk), while another requires ¬At(Spare, Trunk).

Apply it to the spare tire example

Suppose the planner considers these two actions:

- `Remove(Spare, Trunk)`
- `PutOn(Spare, Axle)`

If `PutOn(Spare, Axle)` requires the spare to be out of the trunk and available for installation, the planner must check whether that precondition is satisfied at the preceding state level. It cannot simply execute actions whose preconditions are not met.

Remember: Mutex does not mean that an action is impossible forever. It means that the actions or facts are incompatible at that particular planning level.

Planning and Acting in Nondeterministic Domains

What is a nondeterministic domain?

- In a deterministic domain, an action always produces a predictable result.
- In a nondeterministic domain, an action may produce **one of several possible outcomes**.
- The agent cannot always predict exactly what will happen after performing an action.
- Therefore, the agent must consider multiple possible outcomes while planning.
- The agent should be prepared to respond differently depending on the outcome.

Key idea: The agent plans for possible outcomes, not just one fixed sequence of actions.

Planning and Acting in Nondeterministic Domains

How does the agent work?

1. **Identify the current state:** Understand the present situation.
2. **Choose an action:** Select an action that moves toward the goal.
3. **Observe the outcome:** Check what actually happened after the action.
4. **Update the plan:** Choose the next action based on the observed result.
5. **Repeat:** Continue until the goal is achieved.

Important concepts:

- **Contingency plan:** A plan that specifies what to do for different possible outcomes.
- **Conditional action:** An action selected based on the current situation or observed outcome.
- **Feedback:** Information obtained by observing the result of an action.

Key idea: The agent must be able to adapt its actions when the outcome is uncertain.

Example: Robot Navigating Through a Room

Goal: Reach the destination.
Situation: A robot tries to move forward, but its movement is uncertain. It may move forward successfully or get blocked by an obstacle.

Possible outcomes:

- **Outcome 1:** Robot moves forward successfully.
- **Outcome 2:** Robot is blocked and remains in the same position.

C
ontingency plan:

- If the robot moves forward → Check its position and continue toward the destination.
- If the robot is blocked → Turn left or right, then try moving again.

Conclusion: The robot does not rely on one fixed action sequence. It observes the outcome and chooses the next action accordingly.

Key takeaway: Nondeterministic planning prepares the agent for multiple possible outcomes, while acting involves observing what happens and responding appropriately.

A Case Study

The table & chair painting problem

Given a chair and a table, the goal is to have them of the same color.
Initially we have two cans of paint, but the colors of the paint and of the furniture are unknown.
Only the table is initially in the agent's field of view

Remark:

"... of the same color" does not specify which color
$\Longrightarrow$ "color" implicitly existentially quantified.

A Case Study [cont.] The table & chair painting problem [cont.]

- Initial state:

$Init(Object(Table) \wedge Object(Chair) \wedge Can(C1) \wedge Can(C2) \wedge InView(Table))$

- Goal: $Goal(Color(Chair, c) \wedge Color(Table, c))$
  - recall: in goal, variable c existentially quantified
- Actions:

  $Action(RemoveLid(can),$
  $Precond : Can(can)$
  $Effect : Open(can))$
  $Action(Paint(x, can),$
  $Precond : Object(x) \wedge Can(can) \wedge Color(can, c) \wedge Open(can)$
  $Effect : Color(x, c))$

  c is implicitly universally quantified, and is not part of action's variable list
  (partially observable only: we might not know the color)
- Add an action causing objects to come into view (one at a time):

  $Action(LookAt(x),$
  $Precond : InView(y) \wedge (x \neq y)$
  $Effect : InView(x) \wedge \neg InView(y))$

A Case Study [cont.]

The table & chair painting problem [cont.]

- Partially-Observable Problems:
  need to reason about percepts obtained during action

$\Longrightarrow$ Augment PDDL with percept schemata $Percept(\langle fluent \rangle, Precond : \langle fluents \rangle))$
for each fluent. Ex:

- $Percept(Color(x, c),$
  $Precond : Object(x) \wedge InView(x))$

  "if an object is in view, then the agent will perceive its color"

$\Longrightarrow$ perception will acquire the truth value of $Color(x, c)$, for every x, c

- $Percept(Color(can, c),$
  $Precond : Can(can) \wedge InView(can) \wedge Open(can))$

  "if an open can is in view, then the agent perceives the color of the paint in the can"

$\Longrightarrow$ perception will acquire the truth value of $Color(can, c)$, for every can, c

- Fully-Observable Problems:

$\Longrightarrow$ Percept schemata with no preconditions for each fluent. Ex:

- $Percept(Color(x, c))$

- Sensorless Agent: no percept schema

Hierarchical Planning in Artificial Intelligence

Definition: Hierarchical planning is an AI planning technique that breaks a complex task into smaller, simpler subtasks.

- **High-level task:** Defines the overall goal.
- **Subtasks:** Divide the main task into manageable activities.
- **Primitive actions:** Basic actions that can be executed directly by the agent.
- **Task decomposition:** Repeatedly breaks complex tasks into smaller tasks until executable actions are obtained.
- **Advantage:** Reduces planning complexity and makes plans easier to understand and manage.

Key idea: Solve a big problem by dividing it into smaller problems.

Working of Hierarchical Planning

1. **Identify the goal:** Determine the task to be completed.
2. **Decompose the task:** Break the main task into smaller subtasks.
3. **Decompose further:** Divide subtasks until they become simple, executable actions.
4. **Execute actions:** Perform the primitive actions in a valid order.
5. **Complete the goal:** Finish all required subtasks to achieve the overall objective.

Important terms:

- **Abstract task:** A high-level task that requires further decomposition.
- **Primitive task:** A basic action that can be executed directly.
- **Task hierarchy:** The structure showing how a main task is divided into subtasks.

Key idea: Hierarchical planning moves from a high-level goal to detailed executable actions.

Example: Organizing a College Day

Goal: Organize a college event.

Level 1 — Main Task
- Organize College Day.

Level 2 — Subtasks
- Prepare the event.
- Conduct the event.
- Complete the event.

Level 3 — Primitive Actions
- Prepare the event: Book the venue, invite guests, arrange equipment.
- Conduct the event: Welcome guests, deliver speeches, conduct performances.
- Complete the event: Thank participants, collect feedback, clean the venue.

Execution: Perform the primitive actions in a suitable order to complete the event.

Conclusion: Hierarchical planning simplifies a complex goal by dividing it into smaller tasks and executable actions.

HTN Planning: College Day Example

Definition: Hierarchical Task Network (HTN) planning solves complex problems by decomposing high-level tasks into smaller subtasks until executable actions are obtained.

Task Hierarchy:

- **High-level task:** Organize College Day.
- **Subtask 1 – Prepare Event:** Book venue → Invite guests → Arrange equipment.
- **Subtask 2 – Conduct Event:** Welcome guests → Deliver speeches → Conduct performances.
- **Subtask 3 – Close Event:** Thank participants → Collect feedback → Clean venue.

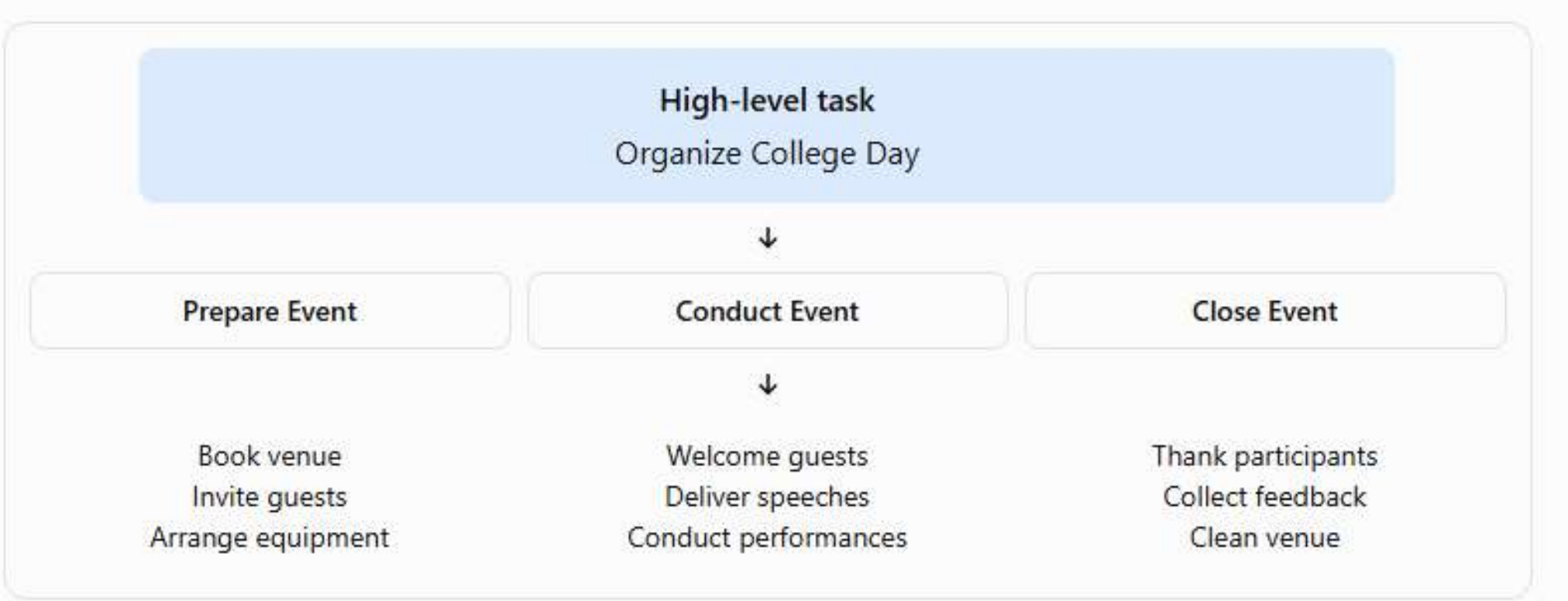
HTN Components:

- **Compound task:** Requires further decomposition.
- **Method:** Defines how a task is decomposed into subtasks.
- **Primitive action:** An executable action that needs no further decomposition.

Final Plan: Execute the primitive actions in the specified order to organize the event successfully.

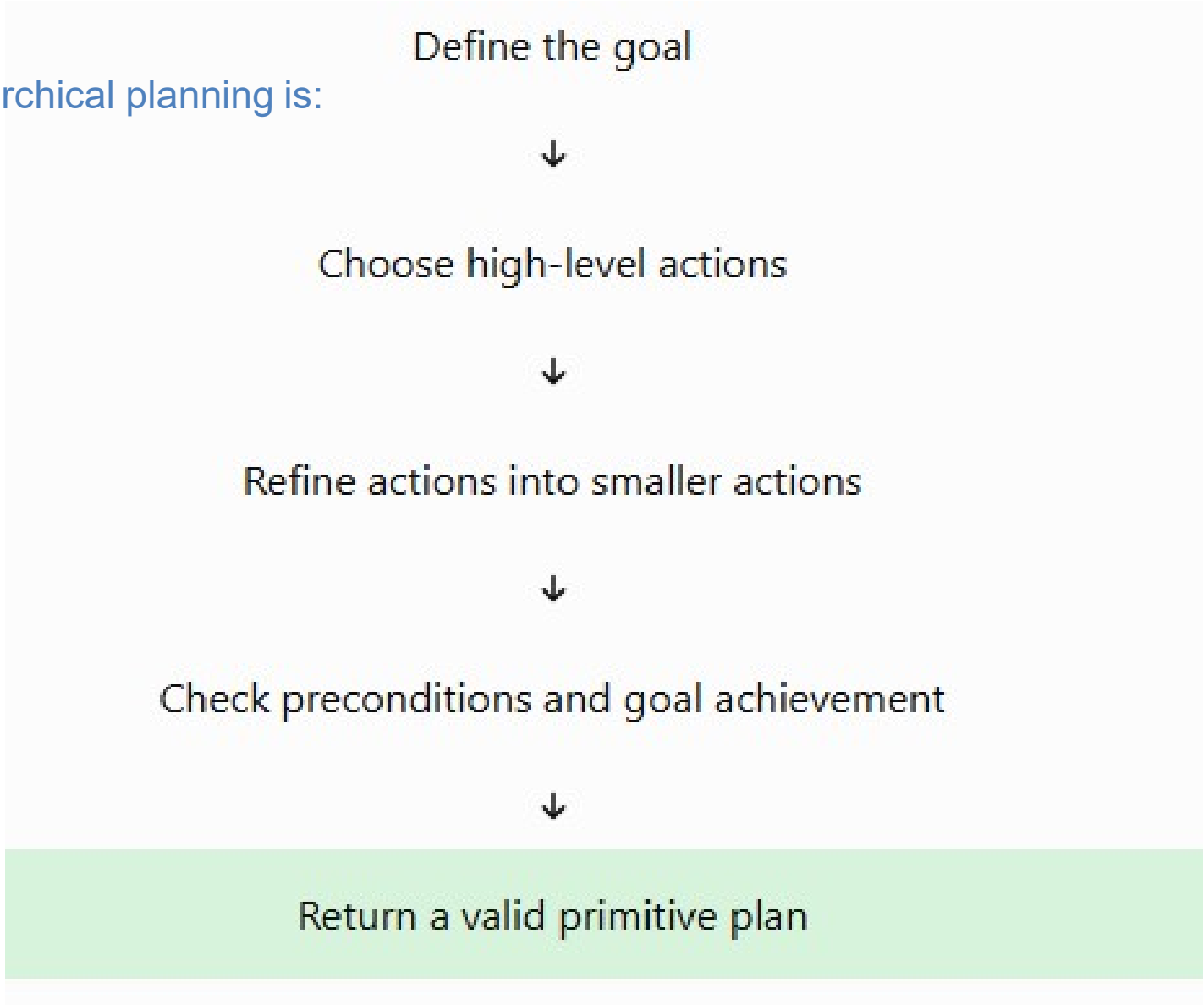
Key Idea: HTN planning transforms a complex goal into an organized hierarchy of manageable tasks and executable actions.

HTN Planning — Task Hierarchy



Explanation: HTN breaks the main task into subtasks and continues decomposing them until the planner obtains primitive actions that can be executed.

The basic process of hierarchical planning is:



Multi-Agent Planning in Artificial Intelligence

Definition: Multi-agent planning is a planning approach in which two or more intelligent agents plan and perform actions to achieve individual or common goals.

- **Multiple agents:** Several agents operate in the same environment.
- **Individual goals:** Each agent may have its own objectives.
- **Coordination:** Agents coordinate their actions to avoid conflicts and improve efficiency.
- **Cooperation or competition:** Agents may work together or compete for resources.
- **Shared environment:** The actions of one agent may affect other agents.

Key idea: Multiple intelligent agents must coordinate their plans when their actions or goals interact.

Working of Multi-Agent Planning

1. **Define goals:** Identify the objectives of each agent and any shared goals.
2. **Allocate tasks:** Assign suitable tasks to different agents.
3. **Plan actions:** Each agent determines the actions required to complete its tasks.
4. **Coordinate plans:** Resolve conflicts, avoid overlapping actions, and manage shared resources.
5. **Execute and adapt:** Agents perform their actions and adjust their plans when conditions change.

Major challenges:

- Coordination and communication.
- Conflicts over shared resources.
- Synchronization of actions.
- Handling unexpected changes.

Key idea: A successful multi-agent plan ensures that individual actions work together to achieve the overall objective.

Example: Multi-Agent Planning in a Warehouse

Goal: Deliver three packages to the dispatch area efficiently.

Agents and tasks:

- **Robot 1:** Collect Package A and deliver it to dispatch.
- **Robot 2:** Collect Package B and deliver it to dispatch.
- **Robot 3:** Collect Package C and deliver it to dispatch.

Coordination:

- Robots plan routes to avoid collisions.
- If two robots need the same narrow passage, they coordinate their movement.
- If a route is blocked, the affected robot selects an alternative route.

Outcome: All three packages reach dispatch safely and efficiently.

Key takeaway: Multi-agent planning coordinates the actions of multiple agents so that they can achieve a common goal without conflicting with one another.